Chair of Machine Learning and Reasoning (Computer Science 6)

RWTH AACHEN UNIVERSITY

Master Thesis

Semantic Partitioning for Partially Observable Deterministic Task and Motion Planning

Hani Alassiri Alhabboub

May 20, 2026

Abstract

This thesis extends Partially Observable Deterministic (POD) planning to Task and Motion Planning (TAMP) under partial observability. Existing TAMP frameworks often assume full observability, or rely on probabilistic models that are expensive to compute. This project addresses the challenge of modeling belief states over object locations in continuous 3D spaces. The central idea is a belief model built from Semantic Boxels—task-relevant cuboids—enabling POD planning to reason about spatial uncertainty without requiring exhaustive discretization. We integrate this abstraction into a PDDLStream-based TAMP framework, using its streams to connect symbolic plans to continuous geometry. The contribution is a simple framework whose Boxel abstraction scales better than uniform voxelization, letting a robot plan under spatial uncertainty—searching for objects it cannot see and then manipulating them.

1 Introduction

We want robots that can carry out everyday tasks on their own—fetching an object, clearing a table—tasks that demand both multi-step reasoning and careful physical manipulation. Task and Motion Planning (TAMP) studies how to do exactly this [6, 9]. TAMP connects a high-level goal (e.g., "fetch the blue cup from the kitchen") to the actual motions the robot must perform to achieve it. TAMP combines discrete task planning with continuous motion planning: symbolic planners generate high-level sequences of abstract actions—what is known as a task plan. However, symbolic planning alone is insufficient for real-world robotic execution. Because the symbolic plan ignores geometry, it can call for actions the robot cannot physically carry out.

Specifically, we must determine whether the proposed symbolic actions are feasible within the robot's continuous geometric and kinematic constraints, and if so, identify the concrete parameters required to execute them—such as object poses, grasp configurations, and collision-free trajectories. For instance, a symbolic instruction like (*pick A*) is only viable if object A is actually reachable, graspable, and a feasible motion exists to carry out the action.

One widely used framework for this is PDDLStream [9]. PDDLStream extends the classical Planning Domain Definition Language (PDDL) [11] by incorporating streams—lazily-evaluated, declarative procedures that can interface with external solvers. These streams are capable of generating or validating continuous parameters on demand, allowing the symbolic planner to retrieve geometric or kinematic information only as needed. This simplifies the coupling between high-level symbolic reasoning (what to do) and low-level geometric feasibility (how to do it), making the planning process both flexible and efficient.

However, the original PDDLStream framework assumes full observability and deterministic action outcomes—assumptions that rarely hold in practice. In practice, robots frequently operate under partial observability, due to occlusions, limited sensor coverage, or dynamically changing environments. In such settings, crucial information—such as the location of a target object or the presence of an obstacle—may be unknown at the start, so the robot has to actively look for it while carrying out the task.

Addressing this limitation is the central focus of this thesis: extending PDDLStream to operate under partial observability. This includes the ability to represent and reason over belief states that capture uncertainty in continuous spatial variables. Naive representations—such as a uniform voxel grid over 3D space—make the belief space blow up in size and quickly become impractical to plan with. A useful belief representation must therefore be detailed enough to be expressive, yet small enough to plan with quickly. The core research question is: *How can we formally define and integrate a belief representation based on semantic partitioning into a POD-TAMP framework?* Here, we present a novel approach to discretizing the workspace around the objects the robot detects and the regions they occlude. The unit of this discretization is the Semantic Boxel: a task-relevant, occlusion-aware cuboid, built from objects' known 3D models, that the robot uses to represent its belief. Unlike a dense, uniform discretization, this provides a structured, scalable way to represent and reason about spatial uncertainty.

This thesis develops a system in which a robot, facing uncertainty about object locations, can plan and execute a sequence of information-gathering and manipulation actions. It reasons over abstract beliefs about which Boxels objects might occupy, using PDDLStream to connect this reasoning to continuous motion and manipulation. Sensing is a symbolic action evaluated against a fixed overhead camera, not a viewpoint the robot moves to; and because the underlying planner is classical, partial observability is handled by optimistic sensing with reactive replanning rather than contingent planning—a sensing action is assumed to succeed, and the

system replans if execution shows otherwise. The result is a framework that makes planning under spatial uncertainty simpler and more reliable. The approach is demonstrated on tabletop hidden-object retrieval and stacking; the partial-observability mechanism is the contribution, and longer task horizons are a natural extension. This thesis is structured as follows: Section 2 provides essential background information on classical planning, reasoning under uncertainty, and specific frameworks like PDDLStream that are foundational to this work. Section 3 then reviews related work in TAMP under partial observability, belief representation, and relevant planning paradigms. Section 4 details the methodology, including the formalization of Boxels through adaptive semantic discretization, the Semantic POD-TAMP model, and the PDDLStream integration. Finally, Section 5 describes the plan for evaluating our framework and concludes by outlining the contributions and significance of the research.

2 Background

This chapter builds up the pieces the thesis depends on, ending with PDDLStream and how 3D space is represented for planning.

2.1 AI Planning Fundamentals

Artificial Intelligence (AI) planning produces a sequence of actions—a plan—that gets from a given initial state to a specified goal [12].

2.1.1 State-Space Models in Classical Planning

The formal basis for classical planning problems is a *state-space model* [10, 12, 16]. A state-space model defines every possible configuration of the world and how actions move between configurations.

More formally, a deterministic state-space model can be described as a tuple $\Pi = \langle S, s_0, S_G, Act, A, f \rangle$, where:

- S: A set representing all possible *states* the world can be in. For example, in a simple blocksworld problem with two blocks A and B and a table, a state could be "Block A is on Block B, and Block B is on the table, and the robot hand is empty."
- $s_0 \in S$: The specific *initial state* from which planning begins. E.g., "Block A is on the table, Block B is on the table, hand empty."
- $S_G \subseteq S$: The set of *goal states*. Any state in this set satisfies the desired objective. E.g., any state where "Block A is on Block B."
- Act: A set of all possible *actions* the agent can perform. E.g., *pick blockX*, *stack blockX on blockY*.

- $A(s) \subseteq Act$: A function that, for any given state $s \in S$, returns the subset of actions from Act that are *applicable* (can be executed) in state s. For example, *pick blockA* is applicable only if Block A is clear (nothing on top of it) and the robot hand is empty.
- $f(a, s) \rightarrow s'$: A deterministic *state transition function*. Given a state s and an applicable action $a \in A(s)$, $f(a, s)$ returns the unique successor state s' that results from executing action a in state s. For example, if s is "A on table, B on table, hand empty" and a is *pick blockA*, then $s' = f(a, s)$ would be "B on table, hand holding A."

A *solution* or *plan* is then a sequence of actions $\sigma = [a_0, a_1, \dots, a_n]$ such that if s_i is the state before action a_i, then $a_i \in A(s_i)$, $s_{i+1} = f(a_i, s_i)$, and the final state s_{n+1} is in S_G. Often, we are interested in an *optimal plan*, which could be the shortest sequence of actions or one that minimizes some associated cost.

Compact Representation: STRIPS, and PDDL Explicitly enumerating all possible states S is infeasible for all but the simplest problems. Therefore, planning problems are typically described in a more compact, *factored* representation using a planning language. In this representation, a state is defined by a set of propositions (or ground atoms) that are currently true.

A foundational language for this is **STRIPS (Stanford Research Institute Problem Solver)** [7]. In STRIPS:

- **States** are represented as sets of true propositions (e.g., `(on A B)`, `(ontable B)`, `(handempty)`).
- **Actions** are defined by:
  - *Preconditions*: A set of propositions that must be true in the current state for the action to be applicable.
  - *Effects*: Specified as two lists:
    - *Add-list*: A set of propositions that become true after the action is executed.
    - *Delete-list*: A set of propositions that become false (are no longer true) after the action.
- The transition function $f(a, s)$ is implicitly defined: the new state s' is obtained by taking state s, removing all propositions in the delete-list of a, and then adding all propositions in the add-list of a.
- The initial state (s_0) is a set of initially true propositions, and the goal (S_G) is specified by a set of propositions that must be true in any goal state.

The **Planning Domain Definition Language (PDDL)** [11] builds upon and extends the STRIPS representation, providing a more expressive and standardized syntax for a wider range of classical planning problems. In PDDL:

- **Domain File** defines:

  - *Types*: Categories for objects (e.g., `block`, `robot`).
  - *Predicates*: Relations or properties (e.g., `(on ?x - block ?y - block)`).
  - *Action Schemas*: Templates for actions, generalizing STRIPS operators. For instance:

```
(:action stack
    :parameters (?obj1 - block ?obj2 - block)
    :precondition (and (holding ?obj1) (clear ?obj2))
    :effect (and (on ?obj1 ?obj2) (not (holding ?obj1))
                 (not (clear ?obj2)) (handempty) (clear
                     ?obj1))
)
```

    In STRIPS terms, this action's delete-list contains `(holding ?obj1)` and `(clear ?obj2)`, while its add-list contains `(on ?obj1 ?obj2)`, `(handempty)`, and `(clear ?obj1)`.

- **Problem File** specifies:
  - *Objects*: Specific objects (e.g., `A, B, C - block`).
  - *Initial State (`Init`)*: Ground atoms true at the start.
  - *Goal (`G`)*: Ground atoms that must be true in a goal state.

A PDDL problem $P = \langle \text{Domain}, \text{Instance} \rangle$ thus compactly encodes the state model $S(P)$. Ground actions (action schemas with parameters replaced by specific objects) form the set Act. More advanced PDDL versions also support features beyond STRIPS, such as conditional effects (`(when <condition> <effect>)`), numeric fluents, and axioms (derived predicates).

2.2 Planning under Partial Observability

Classical planning's assumption of full observability is often violated in robotics. When an agent has incomplete information about the true state of the world, it must reason about its *belief state*.

2.2.1 Belief States and K-Literals

A belief state represents the agent's current set of possible world states consistent with its history of actions and observations. In Partially Observable Deterministic (POD) planning [1, 4], the world itself behaves deterministically, but the agent's knowledge is incomplete. The belief state b is explicitly a set of possible states. Actions transition this set, and observations prune it. A common way to reason about knowledge in POD planning is through K-literals [3, 10]. K-literals make the robot's knowledge about a proposition p explicit. For example:

- $K(p)$ signifies "p is known to be true."
- $K(\neg p)$ signifies "p is known to be false."

If neither $K(p)$ nor $K(\neg p)$ holds, the truth of p is unknown. Separately, p is "possibly true" whenever $K(\neg p)$ does not hold—which also covers the case where p is known to be true.

K-literals are the basis of a *translation* that compiles a partially observable problem into a classical one [10]. Each literal L of the original problem is replaced by the two fluents $K(L)$ and $K(\neg L)$; in the translated problem these are made true in the initial state only where the agent actually knows the corresponding value, and a literal whose value is unknown contributes neither. The translated problem therefore carries no uncertainty in its initial state and can be solved by an ordinary classical planner, which now searches for a plan that reaches a desired *state of knowledge*—for example K(goal_achieved)—rather than a desired world state. Because the pair $K(L)$ and $K(\neg L)$ together encodes whether the agent knows the truth of L, we call such a knowledge fluent a *Know-If Fluent (KIF)*. KIFs are the belief representation used throughout this thesis (Section 4).

2.2.2 Probabilistic vs. Deterministic Partial Observability

POD planning differs from planning with Partially Observable Markov Decision Processes (POMDPs) [18]—a distinction that matters because we later compare our approach to POMDP-based TAMP. In POMDPs, the world is modeled as a Markov Decision Process (MDP) with probabilistic transitions and observations, and the belief state is a probability distribution over world states. While highly expressive, POMDPs are computationally very demanding. POD planning assumes outcomes and observations are deterministic but unknown. This makes it more tractable than POMDPs, and it fits robotic problems where the uncertainty comes from not knowing the world rather than from actions behaving randomly.

2.3 Task and Motion Planning (TAMP)

Task and Motion Planning (TAMP) connects high-level task goals with a robot's actual physical movements [8]. The core problem is ensuring that a plan is physically possible. High-level planners often ignore geometry, leading to plans that seem correct but can't be executed. Yet, including all geometric details from the start makes the problem too complex to solve. Combining high-level logic with real-world geometry is the central difficulty of TAMP, and frameworks like PDDLStream exist to bridge it.

2.3.1 PDDLStream for TAMP

PDDLStream [9] is a TAMP framework built to bridge this symbolic-continuous gap. It extends the classical Planning Domain Definition Language (PDDL) by introducing the core concept of **streams**.

Streams: Declarative Procedures for Continuous Parameters A stream in PDDLStream is a procedure that generates or tests values for continuous parameters required by symbolic actions. These parameters might include object poses, robot configurations, grasp transformations, or motion trajectories. Streams have both a procedural and a declarative component:

- **Procedural Component (Conditional Generator):** This is the actual code (e.g., a Python function) that, given some input values (which can be discrete symbols or other continuous values), produces a (potentially infinite) sequence of output values. For example, an inverse kinematics (IK) stream, given an object, its target pose, and a grasp, would generate a sequence of robot configurations that achieve that end-effector pose. A collision-checking stream, given a trajectory, would test if it's collision-free.
- **Declarative Component (Stream Specification):** This part is described in a PDDL-like syntax and informs the symbolic planner what the stream does, what inputs it requires, what outputs it produces, and, crucially, what logical conditions (fluents) are associated with these inputs and outputs. This specification includes:
  - `:inputs`: The typed parameters the stream procedure takes.
  - `:domain`: A set of PDDL facts (preconditions) that must be true for the stream to be applicable for a given set of inputs. For example, an IK stream might require that a valid pose and grasp for the object are already known.
  - `:outputs`: The typed parameters that the stream procedure generates.
  - `:certified`: A set of PDDL facts that are guaranteed to be true if the stream successfully produces a set of outputs for given inputs. These "certified fluents" are added to the planner's state and represent new knowledge derived from the continuous domain. For instance, if an IK stream successfully finds a configuration `?q` for picking object `?obj` at pose `?p` with grasp `?g`, it might certify the fluent `(KinSolution ?obj ?p ?g ?q)`.

PDDLStream employs a **lazy evaluation** strategy. Streams are not called exhaustively beforehand; instead, the symbolic planner calls them "on-demand" when it encounters an action whose preconditions require a continuous parameter to be instantiated or a continuous condition to be tested.

Example PDDLStream Workflow Consider a symbolic action `(pick ?obj ?p ?g ?q ?t)` (pick object `?obj` from pose `?p` using grasp `?g` via configuration `?q` along trajectory `?t`). For this action to be applicable:

1. A stream might first be called to sample a stable `?p` for `?obj` on a surface. If successful, it certifies `(AtPose ?obj ?p)`.
2. Then, another stream samples a grasp `?g` for `?obj`. If successful, it certifies `(Grasp ?obj ?g)`.
3. An IK stream then takes `?obj, ?p, ?g` as input and tries to find a robot configuration `?q`. If successful, it certifies `(KinSolution ?obj ?p ?g ?q)`.
4. Finally, a motion planning stream takes the current configuration and `?q` as input to find a collision-free trajectory `?t`, certifying `(Motion ?q_start ?t ?q)`.

Only if all these streams succeed and certify their respective fluents will the preconditions of the symbolic `(pick ...)` action be met.

The overall PDDLStream algorithm often reduces a TAMP problem to a sequence of finite PDDL problems, iteratively calling streams to discover more certified fluents until a complete, grounded plan is found. The original PDDLStream framework, however, was primarily designed for fully observable and deterministic settings.

2.4 Spatial Representation for Robotic Planning

A robot needs some way to represent the 3D space around it and the objects in it. Representing space is harder under partial observability, because the robot must also represent what it does not yet know.

2.4.1 Discretization of Continuous Space

Robots operate in continuous space, but symbolic planners reason over discrete states — so the continuous space has to be discretized somehow.

Voxel Grids and Octrees A common approach to represent 3D environments is to discretize them into a grid of uniform volumetric pixels, or **voxels**. Each voxel can store information, such as whether it is occupied, free, or unknown. While straightforward, uniform voxel grids can be memory-intensive and computationally expensive for large environments or high resolutions.

Octrees (Figure 1) [17, 22] offer a more efficient hierarchical data structure for representing 3D space. An octree is a tree data structure in which each internal (non-leaf) node has exactly eight children, while leaf nodes have none. It starts with a single large voxel encompassing the entire space of interest. If this voxel is not homogeneous (e.g., it contains both free and occupied space), it is recursively subdivided into eight smaller child voxels (octants).

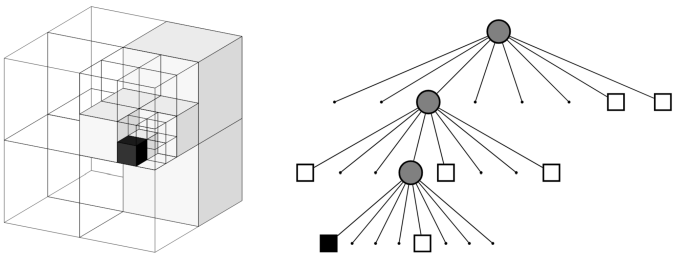

Figure 1: Example of an octree storing free (shaded white) and occupied (black) cells. The volumetric model is shown on the left and the corresponding tree representation on the right.

This subdivision continues until voxels are homogeneous or a minimum resolution/maximum depth is reached. This process creates a tree where large, uniform regions (whether free,

occupied, or unknown) are represented by single nodes at a shallower depth, making it highly efficient for representing sparse environments in comparison to uniform grid voxelization. The **OctoMap** framework [17] uses this principle to build probabilistic 3D occupancy maps from noisy sensor data. In such a framework, unobservable space is typically represented implicitly by the absence of nodes in a given volume; querying such a region returns an ”unknown” status.

Sidd's Critical Regions To make planning more efficient, space can be divided based on task-relevant meaning rather than uniform geometric criteria. One such approach is the concept of **Critical Regions (CRs)**, introduced by Molina et al. [20] and later extended by Shah and Srivastava [24]. Those CRs are learned sub-regions of the configuration space where important features—such as success rates of controllers, object visibility, or transition likelihoods—change significantly between those regions.

In Sidd's approach, these regions are identified using labeled data from motion planning problems, where demonstrations or sampled plans reveal which parts of the space are essential for solving tasks. Once identified, a region-based Voronoi diagram (RBVD) is used to partition the configuration space around the CRs: each point is assigned to the nearest critical region, forming abstract states. Transitions between these abstract states define abstract actions, forming the backbone for hierarchical planning. By focusing computation on these task-relevant regions, this approach allows for more scalable and efficient planning, but is limited to static and pre-defined regions with labeled data.

Our work combines these techniques, as explained in Section 4.

3 Related Work

Integrating task and motion planning with partial observability is challenging, particularly when a robot must act on incomplete information about its environment. This section reviews previous work on this topic, focusing on planning approaches and methods for representing a robot's beliefs about the world.

3.1 TAMP under Partial Observability

Existing approaches to TAMP under partial observability generally either use formal, probabilistic models or more ad-hoc, reactive methods, or a combination of both.

3.1.1 POMDP-based TAMP Solutions

Several works formulate TAMP under uncertainty as a POMDP to formally handle uncertainty and find optimal plans [6, 14, 23]. TAMPURA [6], for example, learns a coarse abstract model of each given controller's preconditions and effects, builds a non-deterministic MDP, and

solves it with an uncertainty-aware solver (LAO*) [15] to produce risk-aware, information-gathering plans. However, a persistent challenge is the representation of belief over continuous properties, particularly object poses. Implementations often resort to particle filters over dense voxel grids to approximate these continuous belief states [6, 13], which does not scale well to large environments.

3.1.2 Partially Grounded Plans in TAMP

In contrast to formal probabilistic models, other TAMP systems handle uncertainty by deferring it to execution [21]. These methods generate partially grounded plans, leaving ”gaps” for actions with unknown outcomes. During execution, specialized reactive controllers attempt to fill these gaps. If a controller fails, the failure is treated as a new constraint, and the system replans. This reactive approach avoids the need to model outcome probabilities explicitly. However, because it does not reason about uncertainty proactively, it can struggle with problems that require multi-step, strategic information gathering, such as deciding which of several actions is most likely to reveal a hidden object.

3.1.3 Modern Approaches to TAMP under Uncertainty

Recent works have leveraged learning and large models to address partial observability, offering different strategies than our own.

One approach adds high-level strategic reasoning on top of standard planners to handle real-world messiness. For instance, Ma et al. [19] first reason backward from a vague goal to decide which objects are important (Task-level Backward Search), and then use a constraint graph to find a safe order to move objects in a cluttered scene (Object Manipulation Constraint Graph). Our approach is more fundamental. Instead of adding a separate strategic layer, we introduce a new geometric belief representation, Boxels. This lets our planner treat which regions are observed versus occluded as first-class planning state, so information-gathering is a core planner action rather than a separate, pre-planning step; the visibility signal is currently supplied by an oracle.

Another approach uses large foundation models to interpret scenes. Zhao et al. [25], for instance, use a Vision-Language Model (VLM) to look at an image and directly answer questions for the planner, like ”Is the drawer empty?”. This replaces traditional perception modules with a black-box model. Our method, in contrast, represents space with a structured, object-centric partition into discrete semantic Boxels, over which the planner reasons symbolically. In the current implementation object poses are supplied by a ground-truth oracle rather than a learned detector, so the contrast with VLM-based perception is at the level of representation and reasoning—a structured, inspectable belief versus a black-box model—not perception itself. Integrating a real detector is left to future work.

Finally, some methods move away from symbolic planning altogether and instead use end-to-end learning. Bai et al. [2], for example, train a system that learns a policy for where to move the camera to get an informative view before acting (Task-Aware View Planning or TAVP). This is fundamentally different from our model-based approach, where looking around is not a learned reflex but an explicit action chosen by the planner because its world model indicates that knowledge is missing. Because our method separates planning from execution, it is more transparent and can be retargeted to new goals without retraining a policy; adapting it to a new environment, however, currently requires re-tuning hardcoded geometric constants.

3.2 Spatial Belief Representation in TAMP

In a typical TAMP cycle the planner (i) reasons symbolically over high-level actions (pick, place, move-to-region), (ii) instantiates them with continuous parameters using a motion planner, (iii) executes the resulting trajectory, observes the environment, and (iv) updates its belief before the next planning step. Every one of these phases depends critically on how we represent spatial belief. While general methods were introduced in Section 2.4, their application in TAMP reveals important limitations.

3.2.1 Limitations of Geometric Discretization for POD-TAMP

Octrees: The efficiency of Octrees makes them a particularly useful tool for 3D mapping. While an Octree framework like OctoMap [17] can represent unobserved space, its primary purpose is to efficiently model occupancy based on noisy sensor measurements. For the task of planning to find hidden objects, it has two key limitations:

1. **Lack of Semantic Structure:** Standard Octree subdivision is purely geometric and based on occupancy variance. It does not create semantically meaningful regions corresponding to, for example, ”the volume occluded by object A”. The planner would have to infer this semantic meaning from the raw geometric data, which is a complex task in itself.
2. **Object Integrity:** The recursive subdivision of an Octree can arbitrarily split a single physical object into multiple voxels at different tree depths. This complicates reasoning about objects as whole entities, which is essential for manipulation planning (e.g., ”pick up object A,” not ”pick up voxels 1, 2, and 5 of object A”).

3.2.2 Limitations of Global Semantic Abstractions

Sidd’s Critical Regions (CRs): The Critical Regions method [24] learns a global set of task-relevant regions, giving the planner a meaningful, non-uniform way to abstract the space. But this partitioning is global and mostly fixed, so it does not work well when objects move. For example, the region ”behind object A” depends on where A currently is and where the robot is standing—a fixed map cannot track that.

4 Approach

This chapter details the methodology for integrating a novel spatial abstraction, Boxels, into a Partially Observable Deterministic (POD) Task and Motion Planning (TAMP) framework built upon PDDLStream.

4.1 Overview

Our methodology links abstract belief representation to continuous geometric reasoning through three components:

1. **Adaptive Semantic Discretization:** A dynamic procedure for partitioning the robot's workspace into a set of meaningful Semantic Boxels, i.e. semantically meaningful subspaces, based on detected objects and their known 3D models.
2. **A Formal POD-TAMP Model:** A formal model that uses these Boxels as the foundation for its state representation, allowing a POD planner to reason about knowledge and uncertainty regarding object locations and unobstructed spaces.
3. **PDDLStream Integration:** An implementation using custom PDDL fluents, actions, and streams that operate on the belief state over Boxels, realizing the formal model in PDDLStream.

4.2 Adaptive Semantic Discretization: Generating Boxels

We build the Boxel abstraction through a process of *adaptive semantic discretization*. This process dynamically generates a set of Semantic Boxels ($\mathcal{BX}_{\text{set}}$) by partitioning the workspace based on detected objects and the occlusions they create. It runs as a Python stage before each call to the planner—and is re-run between actions as the scene changes—rather than as a PDDLStream procedure. Figure 2 illustrates the process.

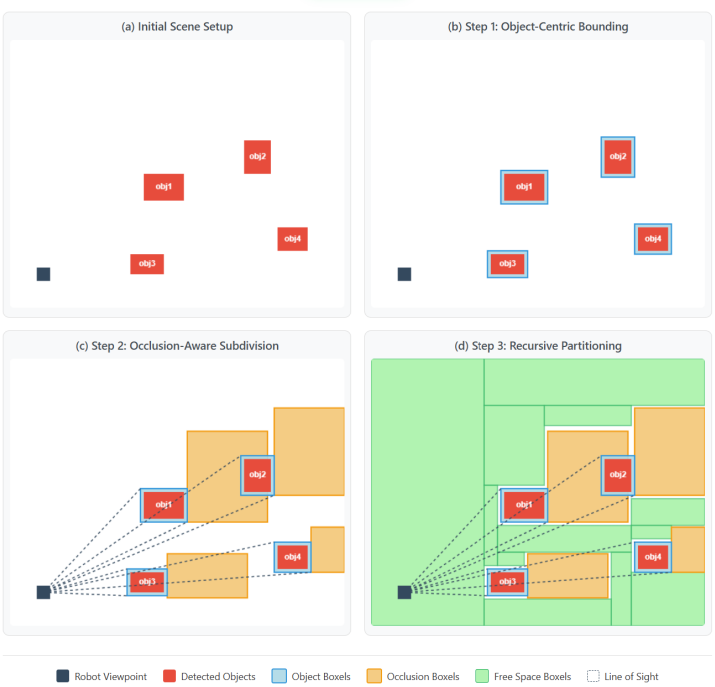

Figure 2: Adaptive Semantic Discretization Process. (a) Initial scene with robot viewpoint and objects. (b) Objects are bounded by dedicated Boxels. (c) Occlusion regions are calculated and defined as new Boxels. (d) Remaining free space is recursively partitioned into Free Space Boxels.

The generation process is as follows:

1. **Object-Centric Bounding:** When an object is detected, we generate a dedicated Boxel: a cuboid that tightly bounds the object's known 3D model at its detected pose. This isolates each known object in its own spatial region. Object detection is provided by an oracle that returns ground-truth object identities and poses from the simulator, which isolates the planning contribution from perception noise; replacing it with a learned, image-based detector is left to future work.
2. **Occlusion-Aware Subdivision:** For each object-bounding Boxel, the space occluded by it from the fixed camera viewpoint is calculated and partitioned into one or more shadow Boxels, split by depth and by any intervening visible obstacles. As the robot acts, the set of shadow Boxels is updated: sensing a region resolves its shadow, and discovering a previously hidden object adds a shadow for that object. A relocated object, however, is not given a new shadow at its destination. Re-creating shadows for moved objects could admit plans that never terminate, so it is deliberately omitted; handling it safely would require additional belief bookkeeping and is left to future work. This explicitly represents regions the robot currently cannot observe, so the planner can target them with information-gathering actions.

3. **Recursive Partitioning:** The space not occupied by object or occlusion Boxels is partitioned recursively, starting from a single Boxel that spans the whole workspace. A Boxel found to be empty is kept as a Free Space Boxel; a Boxel that still overlaps an object or occlusion is split into eight octants, and the procedure repeats on each. Subdivision stops when a region is empty or a minimum Boxel size is reached. The resulting free cells are then merged greedily across shared faces into larger convex Boxels, which reduces their number without changing the represented free volume. This merge is convex-only—two cells are combined only when they share an exactly aligned face—so the partition retains more, smaller free Boxels than a full semantic merge would; this is accepted as sufficient for the object counts evaluated here (see Section 6.1).

This adaptive, object-centric approach contrasts with a uniform voxel grid by focusing discretization effort on task-relevant areas—the objects themselves and the occluded spaces they create—rather than partitioning all space uniformly. Its free-space stage is itself an octree, and a uniform-grid baseline is implemented for comparison. Each generated Boxel is assigned a unique ID and associated geometric data; the completed set of Boxels is supplied to the planner as static facts in its initial state.

4.3 Belief over Boxels with Know-If Fluents

The Semantic Boxels generated in the previous section partition the workspace into a finite set of regions; what the robot does not know is which Boxel a hidden object occupies. We represent this uncertainty with the Know-If Fluents introduced in Section 2. For each object obj and Boxel $Boxel$, the belief carries the K-literal $K(\text{InBoxel}(obj, Boxel))$ when the robot knows the object is in that Boxel and $K(\neg\,\text{InBoxel}(obj, Boxel))$ when it knows the object is not; the absence of both means the object is possibly there. Because the set of Boxels is finite, the belief over object locations is finite as well, and it is held directly in the planner's symbolic state.

Representing belief this way is what keeps planning tractable. As described in Section 2, the K-literal translation compiles a partially observable problem into a classical one. Applied here, it lets an ordinary classical planner reason directly about where objects might be and plan sensing actions to resolve that uncertainty within the same classical search, rather than delegating belief to a separate contingent or belief-space planner.

Knowing which Boxel holds an object is not enough on its own: a manipulation action also needs a reachable grasp and a collision-free trajectory. For this continuous-geometry side we rely on PDDLStream, which extends classical PDDL with *streams*—the lazily-evaluated samplers and tests for continuous parameters described in Section 2. Our approach combines the two in a single PDDL domain: Know-If Fluents carry the discrete belief over Boxels, while streams certify the continuous facts—grasps, inverse-kinematics solutions, and motions—

that the manipulation actions require. A symbolic action such as picking a target object therefore mixes both kinds of precondition: a Know-If Fluent stating that the target's Boxel is known, and stream-certified geometric facts stating that the grasp and motion are feasible. The following two sections make this concrete, giving first the formal POD-TAMP model and then its PDDLStream realization.

4.4 Semantic POD-TAMP Model Definition

We formalize our Semantic POD-TAMP system as an explicit state model, adapting the state-space formulation of Geffner and Bonet [10] to the partially observable deterministic setting. The model is a tuple:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{S}_0, \mathcal{S}_G, \mathcal{A}, f, \mathcal{O} \rangle$$

where:

- $\mathcal{S}$ **(States):** A finite set of abstract states. Each state is a consistent assignment of truth values to a set of atomic propositions $\mathcal{AP}$ built from K-literals (see Section 2). These propositions are grounded in the Boxel representation, e.g., $K\big(\text{InBoxel}\big(obj_k, Boxel_j\big)\big)$.
- $\mathcal{S}_0$ **(Initial States):** The set of possible initial abstract states, representing the initial belief b_0.
- $\mathcal{S}_G$ **(Goal States):** The set of goal states to be reached with certainty, specified by a logical formula over $\mathcal{AP}$ (e.g., K(holding(target_obj))).
- $\mathcal{A}$ **(Actions):** A set of abstract actions defined as PDDLStream action schemas that operate on the belief state, such as 'sense' or 'pick'.
- f **(Transition Function):** A deterministic state-transition function $s' = f(a, s)$ that applies an action's add and delete effects to the current state, so that the successor state reflects the updated knowledge.
- $\mathcal{O}$ **(Sensor Model):** The sensor model relates a sensing action to the observation it yields—the target is found in a Boxel, or it is not—and the resulting belief update. In our system this is not realized as an observation function inside the POD planner: the planner's sensing action is optimistic and assumes the target is found, while the true outcome is obtained at execution time and folded into the belief by a Python sense-plan-act loop, which replans whenever the optimistic assumption proves wrong.

The planning problem is to find a sequence of actions from $\mathcal{A}$ that moves the belief state from $b_0 = \mathcal{S}_0$ to a belief state b_g in which every possible state is a goal state (i.e., $b_g \subseteq \mathcal{S}_G$).

4.5 PDDLStream Integration

This section presents the implemented PDDL domain and streams that realize the model on the belief state over Boxels. For readability the listings show the belief-related fragment of the domain, eliding the geometric and stacking predicates that are not central to this discussion.

4.5.1 PDDL Fluents for Belief Representation

The agent's knowledge is represented directly in the PDDL state. The implemented domain is untyped STRIPS, so object categories are themselves predicates, and the belief over object locations is carried by a single Know-If fluent.

```
(:predicates
    ; --- Category predicates (untyped domain) ---
    (Boxel ?x)               ; ?x is a Boxel
    (Obj ?x)                 ; ?x is a manipulable object

    ; --- Boxel classification ---
    (is_shadow ?b)           ; ?b is an occluded region
    (is_object ?b)           ; ?b bounds a physical object
    (is_free_space ?b)       ; ?b is known-empty space

    ; --- Object location: value and knowledge ---
    (obj_at_boxel ?o ?b)     ; object ?o is at Boxel ?b
    (obj_at_boxel_KIF ?o ?b) ; Know-If fluent: it is known whether ?o is at ?b

    ; --- Other knowledge and robot state ---
    (obj_pose_known ?o)      ; ?o has been localized to a Boxel
    (handempty)              ; the gripper is empty
    (holding ?o)             ; the gripper holds ?o
    ; ... further robot, geometry, and stacking predicates
)
```

Listing 1: PDDL fluents for belief over Boxels

Section 4.3 describes belief with the two K-literals $K(\text{InBoxel})$ and $K(\neg\,\text{InBoxel})$. The implemented domain realizes the same belief with a single Know-If fluent: `obj_at_boxel_KIF` marks that the location of an object relative to a Boxel is *known*, while `obj_at_boxel` carries the value; the absence of the Know-If fluent means the location is still uncertain. The two encodings are logically equivalent, and the single fluent keeps the grounded state smaller.

4.5.2 The Sensing Action

Information gathering is realized by a single `sense` action, which checks whether a target object is present in a given Boxel.

```
(:action sense
    :parameters (?o ?region)
    :precondition (and
        (Obj ?o)
        (Boxel ?region)
        (view_clear ?region)                ; line of sight to the region is clear
        (boxel_fits ?o ?region)             ; ?o physically fits inside ?region
        (not (obj_at_boxel_KIF ?o ?region))  ; only sense while the location is
            unknown
```

```
    )
    :effect (and
        (obj_at_boxel_KIF ?o ?region)        ; the location of ?o is now known
        (obj_at_boxel ?o ?region)            ; optimistic: assume the object is
            found here
        (obj_pose_known ?o)
        (increase (total-cost) 1)
    )
)
```

Listing 2: The implemented sense action

The action is *optimistic*: its effect unconditionally asserts that the object is found in the sensed region. An earlier design used conditional effects that branched on a `found`/`not_found` observation, which would let the planner build a multi-step search ("sense A; if empty, sense B") within one plan. PDDLStream and its classical backend do not support such contingent planning, so the implemented domain commits to the optimistic outcome and relies on *reactive replanning*: when execution reveals that the object is not in the sensed region, the execution loop records the region as empty, updates the belief, and replans. With N candidate regions this converges in at most N replanning cycles and is functionally equivalent to a conditional plan executed in sequence; optimistic planning with replanning on failure is a standard pattern for TAMP under partial observability.

One bounded exception to this observation-driven belief update concerns regions that cannot be observed at all: if the line of sight to a shadow remains blocked after a fixed number of sensing attempts, that shadow is marked empty without ever being directly observed. This keeps the planner progressing when a region is effectively unreachable, but it is unsound—the region could still hold the target. A sound treatment, which would re-ground the view-blocking facts so the planner first clears the obstruction, is left to future work.

The sense action calls no streams. Because a fixed overhead camera observes the scene, no robot sensing configuration has to be sampled, and the only geometric precondition is the derived predicate `(view_clear ?region)`, which holds when no object blocks the line of sight to the region. Streams instead serve the manipulation actions: the domain declares `sample-grasp`, `plan-motion`, `compute-kin`, and `compute-stack-kin`, so that an action such as `pick` requires a valid grasp, an inverse-kinematics solution, and a collision-free motion, each certified by a stream.

5 Evaluation Plan

We evaluate the framework in a simulated robotic manipulation environment, described below.

5.1 Experimental Setup

- **Environment:** We use PyBullet [5] as the physics-based simulation environment, with the 7-DOF Franka Emika Panda as the manipulator.
- **Tasks:** The primary evaluation task is the ”hidden object” scenario, where the robot must find and manipulate a target object occluded by other objects — a task designed to require strategic information-gathering. Two further goals add diversity: multi-cube stacking, and a find-and-tray-stack task in which the robot searches for hidden cubes and stacks them on a tray. We will vary the number of occluders and target objects to test scalability.
- **Perception:** Object detection is performed up front by an oracle that reads ground-truth object poses from the PyBullet simulator, giving exact pose estimates for visible objects and isolating the planning contribution from perception noise. The modeled uncertainty is occlusion uncertainty—which Boxel a hidden object occupies—resolved at run time by a `sense` action backed by raycasting from a fixed overhead camera. RGB and depth images are rendered for visualization but not processed; integrating a learned detector is future work.

5.2 Evaluation Metrics

We adopt success rate and planning time, which permit comparison with TAMPURA's reported numbers [6], alongside boxel-specific compactness metrics. Performance is measured by:

- **Success Rate:** The percentage of problem instances successfully solved within a given time limit.
- **Planning Time:** The wall-clock time required for the planner to find a solution.
- **Replanning and Representation Size:** The number of replans, the number of sense actions, the size of the Boxel set, and the number of facts in the planner's initial state. These capture how the partitioning strategy affects planning load.
- **Scalability:** We will analyze how the above metrics change as the problem complexity increases (e.g., by increasing the number of objects or occluders).

5.3 Baselines for Comparison

The performance of our adaptive Boxel-based POD-TAMP framework will be compared against several baselines to highlight the specific contributions of our approach:

1. **Uniform Voxelization:** A version of our planner that replaces only the adaptive free-space partition with a uniform voxel grid; the object and shadow Boxels are unchanged, isolating the free-space discretization strategy. The cell size is auto-set to the largest object's footprint, since finer cells break placement. This will directly test the scalability benefits of our adaptive approach.

2. **POMDP-based TAMP (TAMPURA):** We compare against TAMPURA [6] using its published per-episode planning times (Table II of the paper), not a re-implementation. This contextualizes the trade-offs between our deterministic-action approach and a fully probabilistic one.

This evaluation tests whether the framework scales and performs well on TAMP under partial observability, especially on tasks that require reasoning about occlusions and information gathering.

5.4 Conclusion

This thesis presented a novel approach to Task and Motion Planning under partial observability by integrating a semantic spatial abstraction, termed Boxels, with a Partially Observable Deterministic planning framework. At the core of the methodology is adaptive semantic discretization: it partitions the workspace into Boxels, concentrating detail on the objects and occluded regions that matter for the task. By modeling belief states over these Boxels, our system lets a PDDLStream-based TAMP framework plan around uncertain object locations and occlusions.

The proposed Semantic POD-TAMP model formalizes this integration, enabling the planner to generate and execute plans that interleave information-gathering and manipulation actions. The use of K-literals to represent knowledge within the PDDL state allows for a direct and conceptually simple integration with existing POD planners. As demonstrated, this enables the robot to reason about what it knows and where uncertainty lies, and to take actions to resolve that uncertainty in a targeted manner.

The expected contribution is a TAMP framework that handles partial observability without enumerating an exhaustive state space, by reasoning over a compact, object-centric abstraction. By replacing exhaustive state-space representations with the Boxel abstraction, this work aims to make TAMP practical in environments where the robot starts out only partially aware of its surroundings. Future work will focus on extending the variety of semantic information used for discretization and exploring more complex, multi-robot planning scenarios.

6 Discussion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

6.1 Limitations and Accepted Simplifications

The system evaluated in this thesis embodies a set of deliberate simplifications. Each was adopted to keep the implementation tractable and to isolate the contribution — semantic Boxel discretization and its integration with partially observable deterministic TAMP — from concerns that are orthogonal to it. This subsection consolidates these simplifications so that the empirical claims of Section 5 are read with the correct scope.

Perception. The scene contains a single fixed overhead camera that observes the whole table. Its rendered RGB-D images are not processed, however: object detection is performed by an oracle that queries ground-truth object poses directly from the simulator and uses ray-casts to determine which objects are visible from the camera's viewpoint. There is therefore no learned detector and no sensor-noise model. This isolates the planning contribution from perception error — a real perception module consuming the camera images could replace the oracle without altering the planning architecture. Because the camera is fixed, the robot never moves to a sensing configuration; this is sufficient for the tabletop scenes studied here, where every shadow is visible from one viewpoint, but a robot-mounted sensor would be required for occlusions not visible from any fixed vantage point.

Manipulation and execution. Grasping is constraint-based: a held object is rigidly welded to the gripper, so objects cannot slip or be dropped in transit. Only top-down grasps are sampled, at a single fixed height offset. The final approach phase of a pick uses a local inverse-kinematics solver that bypasses the planner's collision checks, on the standard TAMP assumption that the short final motion is obstacle-free if the pre-grasp pose is valid. After a place or stack, execution adds a small hardcoded vertical lift before returning; this lift is invisible to the planner — it produces no PDDL state change and moves only along the gravity axis — and exists solely to give the sampling-based motion planner a non-pathological start configuration for the next move. Collision events detected during execution are logged but do not trigger an automatic replan.

Spatial representation. Free-space cells are merged only across exactly aligned shared faces; a full semantic merge — matching view-blocking, observability, and ordering — was not implemented, so the partition contains more Boxels than strictly necessary. When an occluder is relocated, its previously computed shadow geometry and the shadow-blocker map are not recomputed at its new position. This rests on the assumption that the world is static apart from the objects the robot itself moves: nothing else shifts, so the only outdated geometry belongs to the relocated object, whose new pose the planner tracks explicitly and replans from. The symbolic interface also uses string identifiers as proxies for geometric volumes — shadows and moved occluders are passed to the planner as named entities rather than as bounding

boxes or occupancy grids — bridging continuous geometry to discrete planning at the cost of a representation that is not purely geometric.

Planning. The primary goal studied is `holding`; `stack` and `find-and-tray-stack` are shipped extensions that broaden goal diversity. Enabling stacking roughly doubles per-call planning time, traced to the conditional-effects requirement on the pick action; this regression is accepted rather than optimized away, a mitigation having been considered and dropped as out of scope. More fundamentally, when a shadow returns *still-blocked* three times in succession the execution loop marks it as not containing the target and proceeds. The shadow is never directly observed, so for a physically unreachable shadow this violates the invariant that belief should reflect observation. It is accepted to keep the sense–plan–act loop bounded; the run report distinguishes shadows observed empty from shadows given up this way, and the evaluation counts the give-up case as a failure. The principled remedy — re-grounding the planner's view-blocking atoms from current physics — is left to future work (Section 7.1).

Parameters. Several geometric constants — grasp height offsets, approach and lift heights, the octree's minimum leaf size, the camera pose — are tuned for the specific table, robot, and object set used here and would not transfer unchanged to a different setup. Generalizing them, for example by deriving grasp parameters from object geometry, is orthogonal to the core contribution and is noted as future work.

Simplifications disclosed in the approach. Two further simplifications are stated where they are introduced rather than here. The conceptual belief model uses two Know-If literals, $K(p)$ and $K(\neg p)$; the implemented domain collapses these into a single Know-If fluent paired with a value-carrying fluent, an equivalent encoding for the scenarios studied (Section 4.3). The `sense` action is modeled optimistically — it assumes the target is found — with reactive replanning when execution reveals otherwise, because PDDLStream and FastDownward do not support contingent planning (Section 4).

7 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

7.1 Future Work

The simplifications consolidated in Section 6.1 each suggest a concrete next step. The following directions would extend the system beyond the tabletop scenes evaluated here without altering its core architecture.

Learned perception. The most direct extension is to replace the oracle detector with a learned perception module that operates on the rendered RGB-D images rather than ground-truth simulator poses. Because the oracle is confined behind a narrow detection interface, such a module could be substituted without changing the Boxel discretization or the planning layer, turning the architecture's perception-agnosticism from a claim into a demonstrated property.

Active sensing with a robot-mounted camera. The current fixed overhead camera observes the whole table at once. A robot-mounted sensor would require the planner to reason about *where to look*: a sensing-configuration stream that computes a robot pose from which a target region becomes visible, integrated as an additional precondition on the `sense` action. This is the prerequisite for scenes with occlusions not visible from any single fixed viewpoint.

Discovering objects beyond anticipated shadows. When the robot senses a shadow region, any object hidden there is discovered and added to the representation. The scenes studied here are arranged so that every place an object can hide is one such anticipated shadow region. Extending the approach to objects hidden inside concave geometry — such as a container or cupboard, whose interior is itself a hidden region that may hold further objects and occlusions — would require the partitioning to recurse: re-running over the newly revealed interior and recursively bounding whatever it exposes.

Full semantic free-space merge. Free-space cells are presently merged only across exactly aligned shared faces. A full semantic merge that also matches view-blocking, observability, and back-to-front ordering would yield a coarser, more task-relevant partition with fewer Boxels, reducing the planner's initial-state size.

List of References

[1] A. Albore, H. Palacios, and H. Geffner, "A translation-based approach to contingent planning.," in *IJCAI*, vol. 9, 2009, pp. 1623–1628.

[2] Y. Bai, Z. Wang, Y. Liu, W. Chen, Z. Chen, M. Dai, Y. Zheng, L. Liu, G. Li, and L. Lin, "Learning to see and act: Task-aware view planning for robotic manipulation," *arXiv preprint arXiv:2508.05186*, 2025.

[3] B. Bonet, H. Geffner, et al., "Planning under partial observability by classical replanning: Theory and experiments," in *IJCAI Proceedings-International Joint Conference on Artificial Intelligence*, Citeseer, vol. 22, 2011, p. 1936.

[4] B. Bonet and H. Geffner, "Flexible and scalable partially observable planning with linear translations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, 2014.

[5] E. Coumans and Y. Bai, *Pybullet quickstart guide*, `http://pybullet.org`, 2021.

[6] A. Curtis, G. Matheos, N. Gothoskar, V. Mansinghka, J. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, "Partially observable task and motion planning with uncertainty and risk awareness," *arXiv preprint arXiv:2403.10454*, 2024.

[7] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.

[8] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, "Integrated task and motion planning," *Annual review of control, robotics, and autonomous systems*, vol. 4, no. 1, pp. 265–293, 2021.

[9] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, *PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning*, arXiv:1802.08705. Also in Proc. of ICAPS 2020, 2018. arXiv: `1802.08705`.

[10] H. Geffner and B. Bonet, *A concise introduction to models and methods for automated planning*. Morgan & Claypool Publishers, 2013.

[11] M. Ghallab, A. Howe, C. Knoblock, D. McDermott, A. Ram, M. Veloso, D. Weld, and D. Wilkins, "PDDL – the planning domain definition language," AIPS-98 Planning Competition Committee, Tech. Rep. CVC TR-98-003, 1998.

[12] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: theory and practice*. Elsevier, 2004.

[13] N. Gothoskar, M. Ghavami, E. Li, A. Curtis, M. Noseworthy, K. Chung, B. Patton, W. T. Freeman, J. B. Tenenbaum, M. Klukas, V. K. Mansinghka, et al., *Bayes3D: Fast learning and inference in structured generative models of 3d objects and scenes*, arXiv:2312.08715, 2023. arXiv: `2312.08715`.

[14] D. Hadfield-Menell, E. Groshev, R. Chitnis, and P. Abbeel, "Modular task and motion planning in belief space," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2015, pp. 4991–4998.

[15] E. A. Hansen and S. Zilberstein, "Lao*: A heuristic search algorithm that finds solutions with loops," *Artificial Intelligence*, vol. 129, no. 1-2, pp. 35–62, 2001.

[16] T. Hofmann and H. Geffner, "Learning generalized policies for fully observable non-deterministic planning domains," *arXiv preprint arXiv:2404.02499*, 2024.

[17] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous robots*, vol. 34, pp. 189–206, 2013.

[18] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.

[19] Y. Ma, Y. Yuan, S. Wu, and H. Yuan, "A task and motion planning framework for partially observable household manipulation scenes," *Advanced Intelligent Systems*, p. 2 400 897, 2025.

[20] D. Molina, K. Kumar, and S. Srivastava, "Learn and link: Learning critical regions for efficient planning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020.

[21] T. Pan, R. Shome, and L. E. Kavraki, "Task and motion planning for execution in the real," *IEEE Transactions on Robotics*, 2024.

[22] G. Riegler, A. Osman Ulusoy, and A. Geiger, "Octnet: Learning deep 3d representations at high resolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 3577–3586.

[23] M. S. Saleem, R. Veerapaneni, and M. Likhachev, "A pomdp-based hierarchical planning framework for manipulation under pose uncertainty," *arXiv preprint arXiv:2409.18774*, 2024.

[24] N. Shah and S. Srivastava, "Using deep learning to bootstrap abstractions for hierarchical robot planning," in *Proc. of the 21st International Conference on Autonomous Agents and Multiagent Systems (AAMAS '22)*, 2022, pp. 1183–1191.

[25] L. Zhao, W. McClinton, A. Curtis, N. Kumar, T. Silver, L. P. Kaelbling, and L. L. Wong, "Seeing is believing: Belief-space planning with foundation models as uncertainty estimators," *arXiv preprint arXiv:2504.03245*, 2025.